

دليل الذكاء الاصطناعي في PMS

AI Integration — Detailed Guide

شرح تفصيلي: Google Gemini · اقتراح أفكار · توليد مهام · الأمان · التخزين

Laravel 10 · React 19 · gemini-2.5-flash · Multi-Tenant PMS

فهرس المحتويات

1. مقدمة — لماذا AI وكيف يُستخدم في PMS
2. البنية المعمارية — طبقات النظام
3. الإعداد والتكوين (Environment + Config)
4. تكامل Gemini API — الطلب والاستجابة
5. ميزة 1: اقتراح أفكار المشاريع (Project Ideation)
6. ميزة 2: توليد المهام (AI Task Breakdown)
7. مسار «اعتماد الفكرة + إنشاء مشروع + توليد مهام»
8. قوالب الـ Prompts (config/ai.php)
9. الأمان، الصلاحيات، وحدود الاستخدام
10. تخزين البيانات — الجداول والحقول
11. معالجة الأخطاء والحالات الحدية
12. واجهة المستخدم (Frontend)
13. الاختبارات الآلية
14. نقاط للمناقشة في الرسالة العلمية

1. مقدمة — لماذا AI وكيف يُستخدم في PMS

نظام PMS يدمج **Google Gemini** كخدمة خارجية (External API) — أي أن التعلم الآلي لا يعمل داخل السيرفر المحلي. السيرفر يرسل نصاً (prompt) إلى Gemini ويستقبل JSON منظماً، ثم يتحقق منه ويخزّنه في قاعدة البيانات. هذا يجعل الذكاء الاصطناعي **مساعداً للطالب** وليس نظاماً يتحكم تلقائياً بالمشروع.

مبدأ التصميم: Human-in-the-loop — الطالب يقرأ الاقتراحات، يختار، يحفظ، أو ينشئ مشروعاً. المشرف والفريق يرون نشاط التوليد في سجل المشروع والإشعارات. لا يُنشأ مشروع أو مهمة دون إجراء صريح من المستخدم.

ميزتان رئيسيتان:

الميزة	الهدف	المستخدم	المخرجات
Project Ideation	اقتراح 3 أفكار مشاريع تخرج من اهتمامات الطالب	طالب فقط (role:student)	3 بطاقات: اسم، هدف، تقنيات + إمكانية حفظ (bookmark)
AI Task Generation	تقسيم وصف المشروع إلى مهام قابلة للتنفيذ	طالب عضو مقبول في المشروع	10 مهام (قابلة للتعديل) مع estimated_hours و ai_generated=true

2. البنية المعمارية — طبقات النظام

كل طلب AI يمر بنفس النمط المعماري للنظام:

```
React Page (ProjectIdeation.jsx / TasksTab.jsx)
  → apiFetch + Bearer Token (Sanctum)
  → Laravel Route + Middleware (auth, role, throttle)
  → Controller (HTTP validation)
  → Service (AIIdeationService / AITaskService)
  → Http::post → Google Gemini REST API
  → parse JSON + Laravel Validator
  → DB Transaction (tasks / bookmarks / logs)
  → JSON Response → React state update
```

الملفات الأساسية في Backend

الدور	الملف
مفتاح API، نموذج Gemini، endpoint، timeout، retries	config/services.php
قوالب prompts + عدد المهام (AI_TASK_COUNT)	config/ai.php
توليد اقتراحات + logging + bookmarks	AIIdeationService.php
توليد مهام + regenerate + إشعارات + activity	AITaskService.php
endpoints: suggest, bookmarks CRUD 4	AIIdeationController.php
POST /projects/{id}/generate-tasks	AITaskController.php
Rate limiters: ai-ideation , ai-tasks	RouteServiceProvider.php

الملفات الأساسية في Frontend

الدور	الملف
صفحة اقتراح الأفكار + adopt + bookmarks	ProjectIdeation.jsx
بطاقة فكرة: expand، bookmark، adopt & generate	IdeaSuggestionCard.jsx
زر Generate AI Tasks + confirm regenerate	TasksTab.jsx
رابط «Project Ideation» للطالب في الشريط الجانبي	navConfig.js
نصوص *.UI: ideation.*, taskGeneration	ar.js / en.js

3. الإعداد والتكوين

3.1 متغيرات البيئة (env.)

المتغير	الوصف	القيمة الافتراضية
GEMINI_API_KEY	مفتاح Google AI Studio	— (مطلوب)
GEMINI_MODEL	اسم النموذج	gemini-2.5-flash
AI_TASK_COUNT	عدد المهام المطلوبة من AI	10

يُبنى الـ endpoint تلقائياً في `config/services.php`:

`https://generativelanguage.googleapis.com/v1beta/models/{MODEL}:generateContent`

3.2 إعدادات Gemini في `services.php`

المفتاح	القيمة	الاستخدام
timeout	60 ثانية	Http::timeout() — مهلة انتظار الاستجابة
retry_attempts	3	إعادة المحاولة عند فشل الشبكة أو خطأ 5xx
retry delay	ms 500	فاصل بين المحاولات (في Http::retry)

إذا كان `GEMINI_API_KEY` فارغاً، يرفض Service الطلب فوراً برسالة: «AI service configuration error». Contact support — دون استدعاء Gemini.

4. تكامل Gemini API — الطلب والاستجابة

كلا الخدمتين (`AIIdeationService` و `AITaskService`) تستخدمان نفس نمط HTTP:

4.1 رؤوس الطلب (Headers)

```
Content-Type: application/json
x-goog-api-key: {GEMINI_API_KEY}
```

4.2 جسم الطلب (Body)

```
{
  "contents": [
    { "parts": [ { "text": "{PROMPT_BUILT_FROM_TEMPLATE}" } ] },
  ],
  "generationConfig": {
    "temperature": 0.7,
    "maxOutputTokens": 2048 | 4096,
    "responseMimeType": "application/json"
  }
}
```

- **Ideation**: `maxOutputTokens: 2048` — كافٍ لـ 3 اقتراحات JSON.
- **Tasks**: `maxOutputTokens: 4096` — لـ 10 مهام مع أوصاف.
- **responseMimeType**: يفرض على Gemini إرجاع JSON خام (بدون markdown).
- **0.7 temperature**: توازن بين الإبداع والثبات في الاقتراحات.

4.3 استخراج النص من الاستجابة

```
response.json()
  → candidates[0].content.parts[0].text
  → strip optional ```json fences
  → json_decode
  → Laravel Validator (schema rules)
  → return structured array to Controller
```

4.4 منطق إعادة المحاولة (Retry)

يُعاد الطلب تلقائياً (حتى 3 مرات) فقط عند:

- `ConnectionException` — انقطاع شبكة.
- `RequestException` مع `HTTP status ≥ 500` أو بدون `status`.

لا يُعاد عند 401/403/404/422 — هذه أخطاء تكوين أو طلب غير صالح.

4.5 رموز HTTP المُرّجة للواجهة

رسالة للمستخدم	HTTP للعميل	حالة Gemini / Service
success message + data	201 / 200	نجاح
رسالة validation محددة	422	Validation (interests قصيرة، < 20، description regenerate ممنوع)
AI service temporarily» «...unavailable	503	5xx أو ConnectionException
configuration / model unavailable	500	401/403/404 من Gemini
Unable to process AI» «...response	500	JSON غير صالح من AI

5. ميزة 1: اقتراح أفكار المشاريع (Project Ideation)

5.1 مسار المستخدم (UX)

1. الطالب يفتح `/dashboard/ideation` من القائمة الجانبية.
2. يكتب اهتماماته التقنية (10–500 حرف) في حقل multiline.

3. يضغط «Generate» — تظهر رسائل انتظار متدرجة (5s, 30s) أثناء انتظار Gemini.
4. تُعرض 3 بطاقات (IdeaSuggestionCard): اسم، هدف (قابل للتوسيع)، تقنيات.
5. يمكن: حفظ bookmark · إعادة توليد (Regenerate) · اعتماد الفكرة (Adopt & Generate).
6. أسفل الصفحة: قائمة bookmarks المحفوظة مع إمكانية الحذف.

API Endpoints 5.2

الوظيفة	Middleware	Route	Method
توليد 3 اقتراحات	auth · student · throttle:ai-ideation	/api/ai/suggest-projects	POST
حفظ فكرة	auth · student	/api/ai/bookmarks	POST
قائمة bookmarks	auth · student	/api/ai/bookmarks	GET
حذف bookmark (ملكية الطالب فقط)	auth · student	/api/ai/bookmarks/{id}	DELETE

5.3 التحقق في Controller (suggest)

القواعد	الحقل
required · string · min:10 · max:500	interests
max:100 optional array · max:30 كل عنصر	exclude_names

5.4 خط أنابيب generateSuggestions (Service)

1. Check GEMINI_API_KEY + endpoint
2. sanitizeInput(interests):
 - strip_tags → remove <>{}[] → max 500 chars → collapse whitespace
3. assertMinimumInterestsLength (≥ 10 after sanitize)
4. normalizeExcludeNames (trim, unique)
5. buildPrompt from config/ai.php:
 - detect Arabic → language_instruction
 - inject exclude_section if regenerate
6. callGeminiWithRetry → raw JSON text
7. parseAndValidateResponse → exactly 3 suggestions
8. logRequest → ideation_requests (success/error/timeout)
9. return suggestions[] to Controller

5.5 مخطط JSON المطلوب (Ideation)

```
{
  "suggestions": [
    {
      "name": "string (max 100)",
      "goal": "string (max 500)",
      "technologies": ["string (max 50 each)", "..."]
    }
  ]
}
```

قواعد التحقق: suggestions · مطلوب · size:3 · array بالضبط · 1-10 technologies عناصر.

5.6 إعادة التوليد (Regenerate) — exclude_names

عند الضغط على «Regenerate» في الواجهة، يجمع Frontend أسماء الاقتراحات السابقة (previousNames + أسماء الدفعة الحالية) ويرسلها في exclude_names . Service . يُدرجها في ال prompt:

- عربي: «لا تَعد استخدام أو تُقارب أسماء المشاريع التالية: ...»
- إنجليزي: «Do NOT reuse or closely resemble these project names: ...»

الهدف: اقتراحات جديدة مختلفة دون تكرار الأسماء في نفس الجلسة.

5.7 نظام Bookmarks

القاعدة	التفاصيل
الحد الأقصى	10 bookmarks نشطة (archived=false) لكل طالب
عند تجاوز الحد	transaction + lockForUpdate → أقدم bookmark يُؤرشف (archived=true)
التخزين	name, goal, technologies (JSON) في bookmarked_ideas
الرسالة	إذا أُرشف الأقدم: «Your oldest bookmark was archived to make room».

5.8 تسجيل الطلبات — ideation_requests

لا يُخزّن نص الاهتمامات خاماً — فقط interests_hash = SHA-256 للخصوصية والتحليل.

الحقل	الوصف
user_id	الطالب
interests_hash	SHA-256 للنص بعد التنظيف
status	success · error · timeout
response_time_ms	مدة الطلب بالميلي ثانية
error_message	max 255 حرف عند الفشل

6. ميزة 2: توليد المهام (AI Task Breakdown)

6.1 مسار المستخدم

- 1. من تبويب Tasks في تفاصيل المشروع — زر «Generate AI Tasks».
- 2. يُشترط وصف المشروع $20 \leq$ حرفاً (projectDescription من ProjectDetails).
- 3. إذا وُجدت مهام AI سابقاً وكلها pending → يظهر تأكيد SweetAlert لإعادة التوليد.
- 4. إذا بدأ الطالب/الفريق أي مهمة (status ≠ pending) AI → الزر يختفي ولا regenerate.
- 5. بعد النجاح: المهام الجديدة تُدمج في القائمة المحلية؛ المهام اليدوية تبقى.

API 6.2

Method	Route	Body	Middleware
POST	/api/projects/{project}/generate-tasks	{ "regenerate": false true }	auth · student · throttle:ai-tasks · ProjectAccess

AITaskController يتحقق من وجود المشروع و ProjectAccess::canAccess() قبل استدعاء Service.

6.3 قواعد assertGenerationAllowed()

الحالة	regenerate=false	regenerate=true
لا مهام AI	✓ مسموح	✓ مسموح (يُنشئ جديد)
مهام AI موجودة — كلها pending	Set» 422 X «regenerate=true	✓ يحذف القديم ويُنشئ جديد
مهام AI — واحدة على الأقل in_progress/completed	422 X	Cannot regenerate after» 422 X «tasks started

6.4 خط أنابيب generateTasks()

1. Validate description ≥ 20 chars
2. assertGenerationAllowed(project, regenerate)
3. buildPrompt:
 - sanitize title (max 200) + description (max 5000)
 - detect Arabic $\rightarrow$ language_instruction
 - inject task_count from config
4. callGeminiWithRetry
5. parseAndValidateResponse (1..maxTasks tasks)
6. saveTasksTransactionally:
 - if regenerate $\rightarrow$ DELETE tasks WHERE ai_generated=true
 - CREATE new tasks: ai_generated=true, status=pending, estimated_hours
7. recordGenerationActivity $\rightarrow$ ProjectActivity + notifyProjectParticipants
8. logRequest $\rightarrow$ task_generation_logs
9. return { tasks, replaced_count }

6.5 مخطط JSON المطلوب (Tasks)

```
{
  "tasks": [
    {
      "title": "string (max 100)",
      "description": "string (max 500)",
      "estimated_hours": 8
    }
  ]
}
```

قواعد: estimated_hours integer من 1 إلى 40 · عدد المهام من 1 إلى AI_TASK_COUNT (افتراضي 10).

6.6 ما يُخزَّن في جدول tasks

الحقل	قيمة AI
title, description	من Gemini
estimated_hours	من Gemini
ai_generated	true
status	pending
project_id, university_id	من المشروع (multi-tenant)
deadline	null — يحدده الطالب لاحقاً

6.7 ما بعد التوليد — Activity & Notifications

- **ProjectActivity:** action_key = tasksAiGenerated أو tasksAiRegenerated · .meta: count, replaced
- **Notification:** type task.ai_generated لجميع participants (ما عدا الفاعل).
- **Deep link:** /dashboard/projects/{id} في بيانات الإشعار.

7. مسار «اعتماد الفكرة + إنشاء مشروع + توليد مهام»

زر «Adopt & Generate» في IdeaSuggestionCard ينقذ سلسلة من 3 طلبات:

```
Step 1: POST /api/project/create
      { title: suggestion.name, description: suggestion.goal }

Step 2: POST /api/projects/{newId}/generate-tasks
      { regenerate: false }

Step 3: navigate → /dashboard/projects/{id}?tab=tasks

If Step 2 fails → partial success toast → navigate anyway (project exists, no AI tasks)
```

هذا يربط Ideation بـ Task Generation في تجربة واحدة للطالب الذي اختار فكرة جاهزة.

8. قوالب ال Prompts (config/ai.php)

8.1 Ideation Prompt — Placeholders

Placeholder	المصدر
{interests}	نص الطالب بعد sanitize
{exclude_section}	فقرة منع التكرار أو فارغة
{language_instruction}	عربي/إنجليزي حسب كشف Unicode العربي

القالب يطلب «exactly 3 graduation project ideas» ويُلزم JSON فقط بدون markdown.

8.2 Task Breakdown Prompt — Placeholders

Placeholder	المصدر
{language_instruction}	حسب لغة title + description
{project_title}	عنوان منطّف (max 200)
{project_description}	وصف منطّف (max 5000)
{task_count}	من AI_TASK_COUNT (افتراضي 10)

القالب يفرض: مهام متسلسلة · estimated_hours 1-40 · مهمة اختبار/توثيق أخيرة عند الحاجة.

8.3 كشف اللغة (Arabic Detection)

Regex Unicode ranges للحروف العربية في النص المدخل. إذا وُجدت → التعليقات والمخرجات (name/goal أو title/description) بالعربية الفصحى، مع بقاء أسماء التقنيات بالإنجليزية (React, Laravel, MySQL...).

9. الأمان، الصلاحيات، وحدود الاستخدام

9.1 Sanitization (قبل إرسال النص لـ Gemini)

- strip_tags() — إزالة HTML.
- إزالة [] { } < > — تقليل prompt injection.
- حد أقصى للطول: interests 500 · title 200 · description 5000.
- طي المسافات المتعددة.

Rate Limiting 9.2

Route	المفتاح	الحد	Limiter
POST suggest-projects	IP أو user_id	10 / ساعة	ai-ideation
POST generate-tasks	IP أو user_id	10 / دقيقة	ai-tasks

9.3 الصلاحيات

- Ideation + bookmarks: **student only** (role:student middleware)
- Task generation: student + يجب أن يكون ProjectAccess::canAccess (عضو مقبول أو owner).
- Bookmarks: DELETE يتحقق user_id — لا حذف bookmark طالب آخر.

9.4 الخصوصية

نص الاهتمامات لا يُحفظ في DB — فقط hash. وصف المشروع موجود أصلاً في جدول projects (ليس log إضافي لل prompt). مفتاح API على السيرفر فقط — لا يظهر في Frontend.

10. تخزين البيانات — الجداول والحقول

الجدول	الغرض	حقول مهمة
ideation_requests	تحليلات طلبات Ideation	interests_hash, status, response_time_ms
bookmarked_ideas	أفكار محفوظة	suggestion_name, goal, technologies JSON, archived
task_generation_logs	تحليلات توليد المهام	project_id, task_count, status, response_time_ms
tasks (+ migration)	المهام المولدة	estimated_hours, ai_generated, index (project_id, ai_generated)
project_activities	سجل نشاط	tasksAiGenerated / tasksAiRegenerated
notifications	إشعار الفريق	type: task.ai_generated

11. معالجة الأخطاء والحالات الحدية

الحالة	السلوك
Gemini يرجع JSON ناقصاً أو 2 suggestions	Validator fails → 500 generic message
Gemini يلف JSON بـ ``json``	regex strip قبل json_decode
Timeout 60s	status=timeout في 503 · log للمستخدم
Throttle exceeded	Laravel 429 Too Many Requests
Adopt: create OK, generate fails	مشروع موجود · toast تحذير · redirect لـ tasks tab
Regenerate بعد بدء مهمة AI	422 — حماية عمل الفريق

12. واجهة المستخدم (Frontend) — تفاصيل إضافية

ProjectIdeation.jsx

- عداد أحرف `{length}/500` · تعطيل زر Generate إذا `> 10`.
- رسائل انتظار: `takingLonger (30s)` → `stillWorking (5s)` → `generating`.
- `bookmarkNames` Set — تعطيل bookmark مكرر لنفس الاسم.
- تحميل bookmarks عند failure — mount غير blocking.

TasksTab.jsx

- `aiTasks` = `filter ai_generated === true`.
- `canRegenerateAiTasks` = `every ai task status === pending`.
- `showAiGenerateButton` = `hasAiTasks || canRegenerateAiTasks!`.
- عند regenerate: manual tasks تبقى · AI tasks القديمة تُستبدل في الـ UI.

13. الاختبارات الآلية

الملف	ما يختبر
<code>AIIdeationServiceTest</code>	<code>sanitize</code> · <code>parseResponse</code> · <code>bookmark limit</code> · <code>Http::fake</code>
<code>AITaskServiceTest</code>	<code>regenerate rules</code> · <code>parse</code> · <code>transaction</code> · <code>Http::fake</code>
<code>AITaskControllerTest</code>	<code>access</code> · <code>422 validation</code> · <code>201 success</code> · <code>role 403</code>

الاختبارات تستخدم `Http::fake()` لمحاكاة Gemini دون استدعاء API حقيقي.

14. نقاط للمناقشة في الرسالة العلمية

- لماذا Gemini خارجياً؟ — تكلفة، صيانة، وعدم الحاجة لبنية ML محلية في مشروع إدارة مشاريع.
- **Config-driven prompts** — قابلية تعديل السلوك دون deploy كود Service.
- **Strict JSON validation** — طبقة أمان بين LLM غير حتمي وقاعدة بيانات structured.
- **Human-in-the-loop** — AI يقترح؛ الطالب يعتمد؛ regenerate محدود بحالة pending.
- **Multi-tenant** — مهام AI تحمل university_id مثل باقي الكيانات.
- **Observability** — logs (hash, timing, status) دون تخزين prompts كاملة.
- **Rate limits** — حماية من إساءة الاستخدام وتكلفة API.
- **ثنائية اللغة** — كشف تلقائي للعربية في prompt instruction وليس في واجهة منفصلة.

ملخص تدفق كامل (End-to-End)

[Ideation Path]

Student → interests (10-500) → POST /ai/suggest-projects
 → AIIdeationService → Gemini → 3 suggestions → UI cards
 → optional bookmark (max 10) OR adopt → create project → generate-tasks

[Task Path]

Student → project with description (≥20) → POST /projects/{id}/generate-tasks
 → AITaskService → Gemini → N tasks JSON → DB transaction
 → tasks (ai_generated=1) → activity log → team notifications → Tasks tab UI

PMS AI Integration Guide · python docs/generate_pms_ai_guide.py

Companion: PMS-FEATURES-USAGE.pdf · PMS-PROJECT-SCOPE.pdf · PMS-ACTIVITY-DIAGRAM.pdf